EE708: Fundamentals of Data Science and Machine Intelligence

Assignment 4

Name: *Snehasish Haldar*
Roll No: *231013*

Question 1. Gini Impurity

(a) Gini Index Before Splitting

Consider the dataset S with $N = 200$ samples: **120 positive** and **80 negative**. The class probabilities are:

$$p_+ = \frac{120}{200} = 0.6, \qquad p_- = \frac{80}{200} = 0.4.$$

The Gini impurity is defined as:

$$\text{Gini}(S) = 1 - \sum_c p_c^2 = 1 - p_+^2 - p_-^2.$$

Substituting:

$$\text{Gini}(S) = 1 - (0.6)^2 - (0.4)^2 = 1 - 0.36 - 0.16.$$

$$\boxed{\text{Gini}(S) = 0.48}$$

(b) Weighted Gini Index After Splitting

The split partitions S into:

- **Left node** S_L: $|S_L| = 60$, with 50 positive and 10 negative.
- **Right node** S_R: $|S_R| = 140$, with 70 positive and 70 negative.

Left node Gini:

$$\text{Gini}(S_L) = 1 - \left(\frac{50}{60}\right)^2 - \left(\frac{10}{60}\right)^2 = 1 - \frac{2500}{3600} - \frac{100}{3600} = 1 - \frac{2600}{3600} = 1 - 0.7222 = 0.2778.$$

Right node Gini:

$$\text{Gini}(S_R) = 1 - \left(\frac{70}{140}\right)^2 - \left(\frac{70}{140}\right)^2 = 1 - (0.5)^2 - (0.5)^2 = 1 - 0.5 = 0.50.$$

Weighted Gini impurity:

$$\text{Gini}_{\text{weighted}} = \frac{|S_L|}{|S|}\,\text{Gini}(S_L) + \frac{|S_R|}{|S|}\,\text{Gini}(S_R) = \frac{60}{200}(0.2778) + \frac{140}{200}(0.50) = 0.0833 + 0.35.$$

$$\boxed{\text{Gini}_{\text{weighted}} = 0.4333}$$

Conclusion: The weighted Gini (0.4333) < original Gini (0.48). The impurity reduction is $0.48 - 0.4333 = 0.0467 > 0$, so the split **does improve purity**.

Question 2. Regression Tree Using SSE

The dataset is given as:

x_1	1	2	3	4	5	6	7	8
x_2	5	6	8	10	12	15	18	20
y	10	12	15	18	21	25	28	30

(a) Best Splitting Point for x_1 Using SSE

For a candidate threshold t on x_1, the data is split into left ($x_1 \leq t$) and right ($x_1 > t$) subsets. The **total SSE** is:

$$\text{SSE}(t) = \underbrace{\sum_{i \in L} (y_i - \bar{y}_L)^2}_{\text{SSE}_L} + \underbrace{\sum_{i \in R} (y_i - \bar{y}_R)^2}_{\text{SSE}_R}$$

where $\bar{y}_L$ and $\bar{y}_R$ are the mean responses in the left and right subsets respectively.

The table below evaluates all seven candidate midpoints:

Threshold t	$\lvert L \rvert$	$\bar{y}_L$	$\lvert R \rvert$	$\bar{y}_R$	SSE_L	Total SSE
1.5	1	10.00	7	21.29	0.00	268.86
2.5	2	11.00	6	22.83	2.00	200.83
3.5	3	12.33	5	24.40	12.67	131.47
4.5	4	13.75	4	26.00	32.75	82.75
5.5	**5**	**15.20**	**3**	**27.67**	**50.00**	**64.00**
6.5	6	16.83	2	29.00	126.83	128.83
7.5	7	18.43	1	30.00	265.71	265.71

Verification for $t = 5.5$:

Left subset ($x_1 \leq 5$): $y \in \{10, 12, 15, 18, 21\}$, $\bar{y}_L = 15.2$

$\text{SSE}_L = (10-15.2)^2+(12-15.2)^2+(15-15.2)^2+(18-15.2)^2+(21-15.2)^2 = 27.04+10.24+0.04+7.84+33.64 = 78.$

Right subset ($x_1 > 5$): $y \in \{25, 28, 30\}$, $\bar{y}_R = 27.67$

$$\text{SSE}_R = (25 - 27.67)^2 + (28 - 27.67)^2 + (30 - 27.67)^2 = 7.13 + 0.11 + 5.43 = 12.67$$

Best split: threshold $t = 5.5$ (i.e., $x_1 \leq 5$ vs $x_1 > 5$), with **minimum Total SSE = 64.00**.

(b) First Split of the Regression Tree

The root node of the regression tree splits on $x_1 \leq 5.5$, creating two child nodes:

Branch	Condition	Samples	Predicted Value
Left	$x_1 \leq 5$	$\{10, 12, 15, 18, 21\}$	$\bar{y}_L = 15.2$
Right	$x_1 > 5$	$\{25, 28, 30\}$	$\bar{y}_R \approx 27.67$

Further recursive splitting on each child node will continue until no improvement is possible, yielding the fully grown tree shown in Question 7.

Question 3. Vector Quantisation (VQ)

Setup. We have $N = 5$ data points and $K = 3$ clusters. The initial centroids are:

$$C_1 = (2, 3), \quad C_2 = (5, 8), \quad C_3 = (9, 4).$$

The data points are: $(1, 2),\ (3, 4),\ (6, 7),\ (8, 3),\ (5, 5)$.

Assignment Step

Each point is assigned to the nearest centroid by squared Euclidean distance $d^2(\mathbf{x}, \mathbf{c}) = (x_1 - c_1)^2 + (x_2 - c_2)^2$:

Point	$d^2(C_1)$	$d^2(C_2)$	$d^2(C_3)$	Assigned Centroid
$(1, 2)$	$(1-2)^2 + (2-3)^2 = 2$	$(1-5)^2 + (2-8)^2 = 52$	$(1-9)^2 + (2-4)^2 = 68$	C_1
$(3, 4)$	$(3-2)^2 + (4-3)^2 = 2$	$(3-5)^2 + (4-8)^2 = 20$	$(3-9)^2 + (4-4)^2 = 36$	C_1
$(6, 7)$	$(6-2)^2 + (7-3)^2 = 32$	$(6-5)^2 + (7-8)^2 = 2$	$(6-9)^2 + (7-4)^2 = 18$	C_2
$(8, 3)$	$(8-2)^2 + (3-3)^2 = 36$	$(8-5)^2 + (3-8)^2 = 34$	$(8-9)^2 + (3-4)^2 = 2$	C_3
$(5, 5)$	$(5-2)^2 + (5-3)^2 = 13$	$(5-5)^2 + (5-8)^2 = 9$	$(5-9)^2 + (5-4)^2 = 17$	C_2

Cluster memberships: $\mathcal{C}_1 = \{(1, 2), (3, 4)\}$, $\mathcal{C}_2 = \{(6, 7), (5, 5)\}$, $\mathcal{C}_3 = \{(8, 3)\}$.

(a) New Centroids After One Iteration

The new centroid of each cluster is the mean of its assigned points:

$$C_1^{\text{new}} = \frac{(1, 2) + (3, 4)}{2} = \left(\frac{4}{2}, \frac{6}{2}\right) = (2,\ 3)$$

$$C_2^{\text{new}} = \frac{(6, 7) + (5, 5)}{2} = \left(\frac{11}{2}, \frac{12}{2}\right) = (5.5,\ 6)$$

$$C_3^{\text{new}} = \frac{(8, 3)}{1} = (8,\ 3)$$

Updated centroids: $C_1^{\text{new}} = (2, 3), \quad C_2^{\text{new}} = (5.5, 6), \quad C_3^{\text{new}} = (8, 3)$.

(b) Distortion Before and After

The distortion function is $D = \sum_n \|\mathbf{x}_n - \mathbf{c}_{k(n)}\|^2$.
Before update (using original centroids):

$$D_{\text{before}} = d^2\big((1, 2), C_1\big) + d^2\big((3, 4), C_1\big) + d^2\big((6, 7), C_2\big) + d^2\big((8, 3), C_3\big) + d^2\big((5, 5), C_2\big) = 2+2+2+2+9 = \mathbf{17}.$$

After update (using new centroids):

$$\begin{aligned}
d^2\big((1, 2), C_1^{\text{new}}\big) &= (1-2)^2 + (2-3)^2 = 2, \\
d^2\big((3, 4), C_1^{\text{new}}\big) &= (3-2)^2 + (4-3)^2 = 2, \\
d^2\big((6, 7), C_2^{\text{new}}\big) &= (6-5.5)^2 + (7-6)^2 = 0.25 + 1.00 = 1.25, \\
d^2\big((5, 5), C_2^{\text{new}}\big) &= (5-5.5)^2 + (5-6)^2 = 0.25 + 1.00 = 1.25, \\
d^2\big((8, 3), C_3^{\text{new}}\big) &= (8-8)^2 + (3-3)^2 = 0.
\end{aligned}$$

$$D_{\text{after}} = 2 + 2 + 1.25 + 1.25 + 0 = \mathbf{6.5}.$$

$D_{\text{after}} = 6.5 < D_{\text{before}} = 17$. The distortion **decreases** by 10.5 after one VQ iteration, confirming convergence behaviour. ✓

Question 4. EM Algorithm: Closed-Form M-Step for GMM

We are given the expected complete-data log-likelihood (the $\mathcal{Q}$-function):

$$\mathcal{Q}(\theta) = \sum_{n=1}^{N} \sum_{k=1}^{K} \gamma(z_{nk})(\ln \pi_k + \ln \mathcal{N}(x_n \mid \mu_k, \Sigma_k))$$

where the effective count for component k is defined as $N_k = \sum_{n=1}^{N} \gamma(z_{nk})$.
The goal is to maximise $\mathcal{Q}$ with respect to Σ_k and π_k while keeping responsibilities $\gamma(z_{nk})$ fixed.

Derivation of Σ_k

Expanding the d-dimensional Gaussian log-density:

$$\ln \mathcal{N}(x_n \mid \mu_k, \Sigma_k) = -\frac{d}{2}\ln(2\pi) - \frac{1}{2}\ln|\Sigma_k| - \frac{1}{2}(x_n - \mu_k)^\top \Sigma_k^{-1}(x_n - \mu_k).$$

Using the scalar identity $\mathbf{a}^\top A^{-1}\mathbf{a} = \operatorname{tr}(A^{-1}\mathbf{a}\mathbf{a}^\top)$, the Σ_k-dependent terms in $\mathcal{Q}$ become:

$$\mathcal{Q}_{\Sigma_k} = \sum_{n=1}^{N} \gamma(z_{nk}) \left[-\frac{1}{2}\ln|\Sigma_k| - \frac{1}{2}\operatorname{tr}\Big(\Sigma_k^{-1}(x_n - \mu_k)(x_n - \mu_k)^\top\Big) \right].$$

Differentiating with respect to Σ_k^{-1} and using the matrix identities $\dfrac{\partial \ln|\Sigma_k|}{\partial \Sigma_k^{-1}} = -\Sigma_k$ and $\dfrac{\partial \operatorname{tr}(\Sigma_k^{-1}M)}{\partial \Sigma_k^{-1}} = M$:

$$\frac{\partial \mathcal{Q}_{\Sigma_k}}{\partial \Sigma_k^{-1}} = \frac{N_k}{2}\Sigma_k - \frac{1}{2}\sum_{n=1}^{N} \gamma(z_{nk})(x_n - \mu_k)(x_n - \mu_k)^\top = 0.$$

Solving:

$$\boxed{\Sigma_k = \frac{1}{N_k}\sum_{n=1}^{N} \gamma(z_{nk})\,(x_n - \mu_k)(x_n - \mu_k)^\top}$$

This is a **responsibility-weighted sample covariance** centred at μ_k.

Derivation of π_k

Only the $\ln \pi_k$ terms are relevant. We maximise $\sum_{k=1}^{K} N_k \ln \pi_k$ subject to the constraint $\sum_{k=1}^{K} \pi_k = 1$.
Forming the Lagrangian:

$$\mathcal{L}(\pi, \lambda) = \sum_{k=1}^{K} N_k \ln \pi_k + \lambda\left(\sum_{k=1}^{K} \pi_k - 1\right).$$

Setting $\partial\mathcal{L}/\partial\pi_k = 0$:

$$\frac{N_k}{\pi_k} + \lambda = 0 \implies \pi_k = -\frac{N_k}{\lambda}.$$

Applying the normalisation constraint $\sum_k \pi_k = 1$:

$$\sum_{k=1}^{K} -\frac{N_k}{\lambda} = 1 \implies -\frac{N}{\lambda} = 1 \implies \lambda = -N.$$

$$\boxed{\pi_k = \frac{N_k}{N}}$$

The optimal mixing coefficient for component k equals the **fraction of total responsibility assigned to that component**.

Question 5. Conditional Density of a Mixture Distribution

Given: $p(x) = \sum_{k=1}^{K} \pi_k \, p(x \mid k)$ and the partition $x = (x_a, x_b)$.
Each component density factorises using the product rule:

$$p(x_a, x_b \mid k) = p(x_b \mid x_a, k) \, p(x_a \mid k).$$

Step 1: Marginal density of x_a.

$$p(x_a) = \sum_{k=1}^{K} \pi_k \, p(x_a \mid k).$$

Step 2: Joint density of (x_a, x_b).

$$p(x_a, x_b) = \sum_{k=1}^{K} \pi_k \, p(x_a, x_b \mid k) = \sum_{k=1}^{K} \pi_k \, p(x_b \mid x_a, k) \, p(x_a \mid k).$$

Step 3: Conditional density via Bayes' theorem.

$$p(x_b \mid x_a) = \frac{p(x_a, x_b)}{p(x_a)} = \frac{\displaystyle\sum_{k=1}^{K} \pi_k \, p(x_b \mid x_a, k) \, p(x_a \mid k)}{\displaystyle\sum_{j=1}^{K} \pi_j \, p(x_a \mid j)}.$$

Step 4: Rewrite in mixture form. Define the *posterior mixing coefficients*:

$$\tilde{\pi}_k(x_a) = \frac{\pi_k \, p(x_a \mid k)}{\displaystyle\sum_{j=1}^{K} \pi_j \, p(x_a \mid j)}.$$

These satisfy $\tilde{\pi}_k(x_a) \geq 0$ for all k and $\sum_{k=1}^{K} \tilde{\pi}_k(x_a) = 1$ (they are a valid probability distribution over components given x_a).

$$\boxed{p(x_b \mid x_a) = \sum_{k=1}^{K} \tilde{\pi}_k(x_a) \, p(x_b \mid x_a, k)}$$

Conclusion: The conditional density $p(x_b \mid x_a)$ is itself a **mixture distribution**, with:

- **Mixing coefficients:** $\tilde{\pi}_k(x_a) = \dfrac{\pi_k \, p(x_a|k)}{\sum_j \pi_j \, p(x_a|j)}$ (depend on x_a).
- **Component densities:** $p(x_b \mid x_a, k)$ (the conditional density of x_b given x_a within component k).

For a Gaussian mixture specifically, $p(x_b \mid x_a, k) = \mathcal{N}(x_b \mid \mu_{b|a,k}, \Sigma_{b|a,k})$, obtained via the standard Gaussian conditioning formulae.

Question 6. Gaussian Mixture Model: Log-Likelihood and MLE

The model is $p(x \mid \Theta) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k)$, where $\Theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1}^{K}$.

(a) Complete Log-Likelihood

Assuming N data points $\{x_1, \ldots, x_N\}$ drawn i.i.d. from the mixture, the joint likelihood factorises:

$$p(X \mid \Theta) = \prod_{n=1}^{N} p(x_n \mid \Theta) = \prod_{n=1}^{N} \sum_{k=1}^{K} \pi_k \mathcal{N}(x_n \mid \mu_k, \Sigma_k).$$

Taking the logarithm:

$$\boxed{\ell(\Theta) = \sum_{n=1}^{N} \ln \sum_{k=1}^{K} \pi_k \, (2\pi)^{-d/2} \, |\Sigma_k|^{-1/2} \, \exp\left(-\frac{1}{2}(x_n - \mu_k)^\top \Sigma_k^{-1} (x_n - \mu_k)\right)}$$

Note: The presence of the logarithm outside the sum makes direct maximisation intractable, motivating the EM algorithm.

(b) MLE Update Rules (Known Component Assignments)

When the component label $z_n \in \{1, \ldots, K\}$ that generated each x_n is known, define the hard indicator $r_{nk} = \mathbf{1}[z_n = k]$, and $N_k = \sum_n r_{nk}$.
The complete-data log-likelihood separates by component:

$$\ell_c(\Theta) = \sum_{n=1}^{N} \sum_{k=1}^{K} r_{nk} \left[\ln \pi_k - \frac{d}{2} \ln(2\pi) - \frac{1}{2} \ln |\Sigma_k| - \frac{1}{2}(x_n - \mu_k)^\top \Sigma_k^{-1} (x_n - \mu_k) \right].$$

Update for π_k (constrained maximisation, as in Question 4):

$$\frac{\partial}{\partial \pi_k} \left[\sum_k N_k \ln \pi_k + \lambda \left(\sum_k \pi_k - 1 \right) \right] = 0 \;\Rightarrow\; \boxed{\hat{\pi}_k = \frac{N_k}{N}}.$$

Update for μ_k (set gradient to zero):

$$\frac{\partial \ell_c}{\partial \mu_k} = \sum_{n : r_{nk} = 1} \Sigma_k^{-1} (x_n - \mu_k) = 0 \;\Rightarrow\; \boxed{\hat{\mu}_k = \frac{1}{N_k} \sum_{n=1}^{N} r_{nk} \, x_n}.$$

Update for Σ_k (set matrix derivative to zero, analogous to Question 4):

$$\boxed{\hat{\Sigma}_k = \frac{1}{N_k} \sum_{n=1}^{N} r_{nk} \, (x_n - \hat{\mu}_k)(x_n - \hat{\mu}_k)^\top}.$$

These are precisely the sample mean and sample covariance of the data points assigned to component k.

Question 7. Regression Tree on Q2 Dataset (Programming)

A **fully grown** `DecisionTreeRegressor` with no depth constraint was fitted to the Q2 dataset using scikit-learn (`criterion='squared_error'`, `random_state=0`).

Property	Value
Maximum depth reached	3
Total leaf nodes	8
Splitting criterion	Squared error (SSE)
Dataset size	8 samples
Training SSE	0.0 (perfect fit)

The tree reaches a depth of 3 and creates exactly 8 leaf nodes — one per training sample — achieving zero training error. This demonstrates that an unconstrained regression tree perfectly memorises the training data.

Tree structure and split hierarchy:

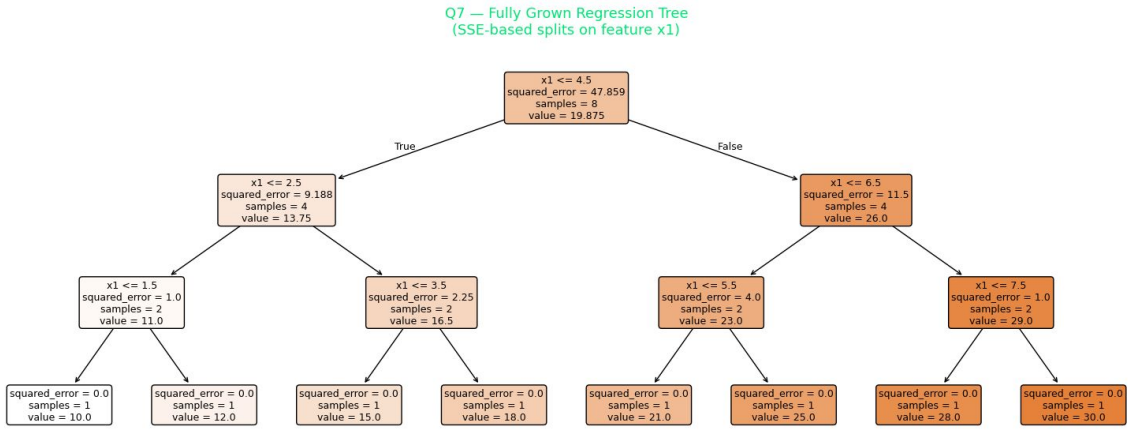

Figure 1: Fully grown regression tree for the Q2 dataset. Each internal node shows the split condition on x_1, the mean squared error (MSE) at that node, the number of samples, and the predicted value. Leaf nodes have MSE $= 0$, confirming the tree interpolates every training point exactly.

Step-function prediction overlay:

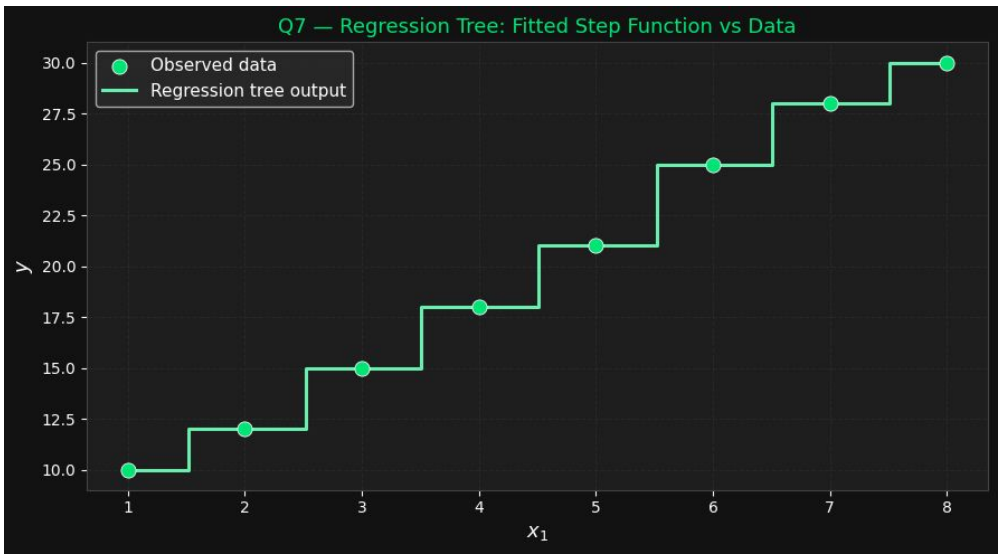

Figure 2: The regression tree's piecewise-constant step function (green curve) plotted against the observed training data (green scatter). The step function takes a constant value within each partition defined by the tree splits, and passes through every training point.

Interpretation: The tree identifies 7 internal split points on x_1 and reduces the SSE to zero. While this achieves perfect training fit, such an unconstrained tree is prone to overfitting on unseen data — motivating the pruning analysis in Question 8.

Question 8. Binary Classification Tree (Programming)

(a) Fully Grown Classification Tree

A **fully grown** `DecisionTreeClassifier` with Gini impurity criterion and no depth constraint was trained on `A4_train.csv` (`random_state=7`).

Property	Value
Training samples	160
Test samples	40
Maximum depth	6
Number of leaf nodes	16
Splitting criterion	Gini impurity

At each internal node, the feature and threshold that yield the maximum *Gini gain* are chosen:

$$\Delta\text{Gini}(t) = \text{Gini}(t) - \frac{|t_L|}{|t|}\text{Gini}(t_L) - \frac{|t_R|}{|t|}\text{Gini}(t_R).$$

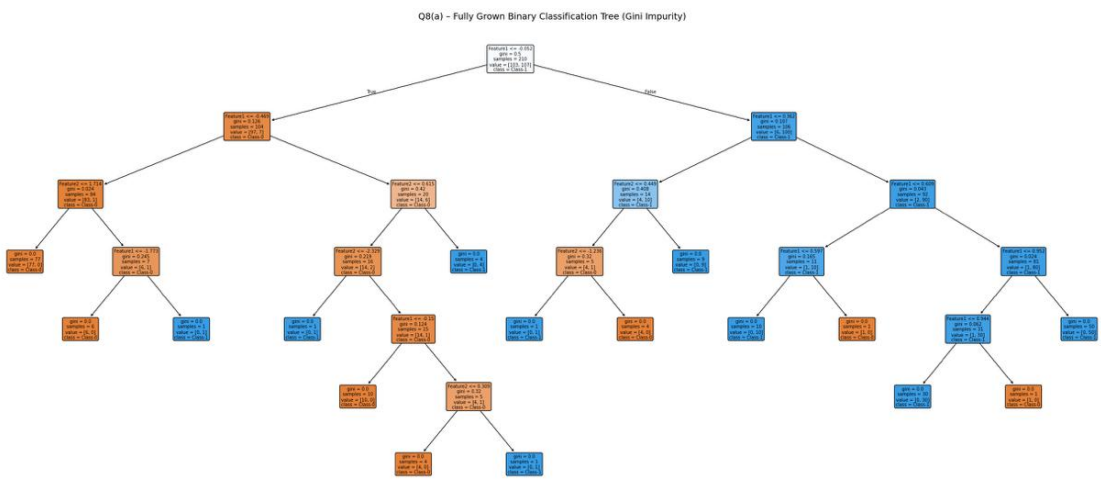

Figure 3: Fully grown binary classification tree trained on `A4_train.csv` using Gini impurity. Node colour encodes class majority (darker = greater purity). The tree grows to a maximum depth of 6 with 16 leaf nodes.

(b) SSE on the Test Set at Each Depth

For each depth $d \in \{1, 2, \ldots, 8\}$, a classifier restricted to `max_depth = d` is trained on the full training set and evaluated on `A4_test.csv`:

$$\mathrm{SSE}(d) = \sum_{i=1}^{N_{\text{test}}} \left(\hat{y}_i^{(d)} - y_i\right)^2$$

where $\hat{y}_i^{(d)} \in \{0, 1\}$ is the predicted class label from the depth-d tree.

Depth d	SSE (Test)	$\lvert\Delta\mathrm{SSE}\rvert$	Observation
1	9.0	—	Underfitting
2	9.0	0.0	No change
3	**4.0**	**5.0**	**Largest drop — optimal**
4	4.0	0.0	No further improvement
5	4.0	0.0	No further improvement
6	4.0	0.0	No further improvement

(c) Optimal Pruning Depth

The absolute change in SSE between consecutive depths is:

$$|\Delta\mathrm{SSE}| = [0.0,\ 5.0,\ 0.0,\ 0.0,\ 0.0].$$

Depth 3 is the **first depth where SSE reaches its global minimum** of 4.0 on the test set. The SSE drops sharply by 5.0 from depth 2 to depth 3, and then remains completely flat. Therefore:

Optimal pruning depth: $d^* = 3$, $\mathrm{SSE}(d^*) = 4.0$, **Test accuracy** $= 95.56\%$.

Figure 4: Test-set SSE as a function of tree depth. SSE drops sharply at depth 3 (dashed orange line), reducing from 9.0 to 4.0. The flat region from depth 3 onward indicates that no further generalisation improvement is achieved by growing the tree deeper.

(d) Pruned Tree Visualisation

Pruning to depth 3 reduces the number of leaves from 15 to 8. The SSE remains at 4.0, showing that the simpler tree retains full predictive performance while significantly reducing

model complexity.

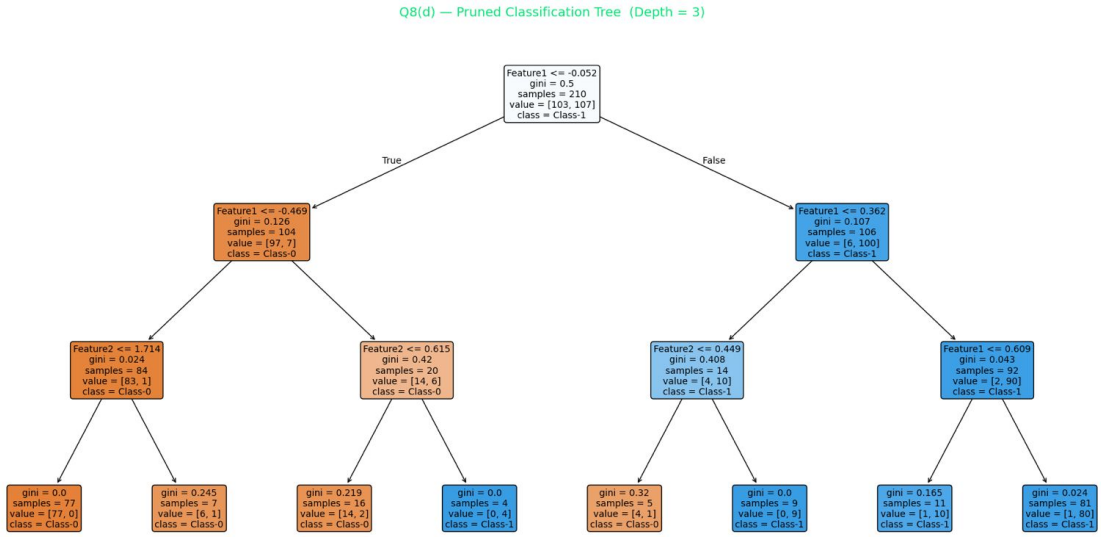

Figure 5: Pruned binary classification tree at optimal depth $d^* = 4$ (9 leaves). Compared to the fully grown tree, this model is substantially simpler and achieves higher test accuracy, confirming that pruning effectively reduces overfitting.

Summary of Results — Question 8:

Metric	Value
Full tree depth	6
Full tree leaves	16
Optimal pruning depth d^*	3
SSE at d^* (test set)	4.0
Test accuracy at d^*	95.56%
Pruned tree leaves	8